

OOPs Interview Questions & Answers

Q: Can we overload constructors in Java?

A: Yes, constructors can be overloaded by defining multiple constructors with different parameter lists.

Q: Can we override constructors in Java?

A: No, constructors cannot be overridden. Only methods can be overridden.

Q: Can we inherit constructors?

A: No, constructors are not inherited. A subclass must define its own constructors.

Q: Can we make a constructor private?

A: Yes, this is used in Singleton design pattern or to restrict object creation.

Q: Can we make a constructor static?

A: No, constructors cannot be static.

Q: Can we make a constructor final?

A: No, constructors cannot be final.

Q: Can we make a constructor abstract?

A: No, constructors cannot be abstract.

Q: Can we overload main method in Java?

A: Yes, we can overload the main method, but only public static void main(String[] args) is entry point.

Q: Can we override main method in Java?

A: No, because main is static and static methods cannot be overridden.

Q: Can we overload static methods?

A: Yes, static methods can be overloaded like normal methods.

Q: Can we override static methods?

A: No, static methods cannot be overridden, but they can be hidden.

Q: Can we overload private methods?

A: Yes, private methods can be overloaded within the same class.

Q: Can we override private methods?

A: No, private methods cannot be overridden as they are not visible to subclasses.

Q: Can we overload final methods?

A: Yes, final methods can be overloaded.

Q: Can we override final methods?

A: No, final methods cannot be overridden.

Q: Can we overload instance methods?

A: Yes, instance methods can be overloaded.

Q: Can we override instance methods?

A: Yes, instance methods can be overridden in subclasses.

Q: What is the difference between overloading and overriding?

A: Overloading: same method name, different parameter list. Overriding: same method signature in subclass.

Q: Why is the main method static?

A: Because it must be invoked by the JVM without creating an object.

Q: Can a static method access instance variables?

A: No, static methods cannot directly access instance variables without an object.

Q: Can a static method call another static method?

A: Yes, static methods can call other static methods.

Q: Can a static method be inherited?

A: Yes, but it cannot be overridden, only hidden.

Q: Can a static method use 'this' keyword?

A: No, because 'this' refers to an instance, and static methods don't have one.

Q: Can we achieve runtime polymorphism using static methods?

A: No, static methods are resolved at compile time.

Q: What is early binding and late binding in Java?

A: Early binding: compile-time (overloading, static). Late binding: runtime (overriding, dynamic dispatch).

Q: What is dynamic method dispatch?

A: It is the mechanism by which a call to an overridden method is resolved at runtime.

Q: Can we achieve runtime polymorphism with data members?

A: No, data members cannot be overridden, only hidden.

Q: What is the difference between method overloading and method overriding?

A: Overloading is compile-time, overriding is runtime. Overloading has different parameter lists.

Q: What is multiple inheritance?

A: When a class inherits from more than one class. Java does not support it with classes, but supports via interfaces.

Q: Why doesn't Java support multiple inheritance with classes?

A: To avoid ambiguity (diamond problem).

Q: Does Java support multiple inheritance with interfaces?

A: Yes, a class can implement multiple interfaces.

Q: What is hybrid inheritance?

A: A mix of two or more types of inheritance (not directly supported in Java).

Q: What is encapsulation?

A: Encapsulation is bundling data and methods that operate on the data into a single unit (class).

Q: How do we achieve encapsulation in Java?

A: By making fields private and providing public getters/setters.

Q: What is the difference between abstraction and encapsulation?

A: Abstraction hides implementation details, encapsulation hides data.

Q: Can we make a class immutable?

A: Yes, by making fields private and final, and not providing setters.

Q: What is inheritance?

A: Inheritance allows a class to acquire properties and behavior from another class.

Q: What is the difference between interface and abstract class?

A: Abstract class can have concrete methods and state, interface only declares behavior (before Java 8).

Q: Can an abstract class have a constructor?

A: Yes, it is used to initialize common fields of subclasses.

Q: Can an interface have a constructor?

A: No, interfaces cannot have constructors.

Q: Can we create object of abstract class?

A: No, abstract classes cannot be instantiated directly.

Q: Can we create object of interface?

A: No, interfaces cannot be instantiated directly.

Q: *What is diamond problem?*

A: An ambiguity problem in multiple inheritance where a method is inherited from multiple classes.